

Assignment 1

Accompanying this assignment you will find an archive file `gtiit-sudoku.zip`. The zip contains a directory `gtiit-sudoku`, with a BlueJ Java project for the GTIIT Sudoku application.

Sudoku is a popular logic-based number placement puzzle. It is played on a 9×9 grid that is subdivided into nine smaller 3×3 boxes.

The objective of the puzzle is to fill the grid with digits from 1 to 9, following these simple rules:

1. Each **row** must contain all digits from 1 to 9 exactly once.
2. Each **column** must contain all digits from 1 to 9 exactly once.
3. Each of the nine 3×3 **boxes** in the board must contain all digits from 1 to 9 exactly once.

A puzzle begins with a number of cells already filled in (the “givens” of “fixed cells”). The challenge is to use pure logical deduction to fill in all the remaining empty cells without breaking any of the three rules.

Project *GTIIT Sudoku* is a text-based implementation of the Sudoku game. It consists of four classes:

- **SudokuCell**: it is the simplest of the classes. It models a cell of the Sudoku, with its possible states: fixed or not fixed, and its contents (value), with zero representing the cell being empty.
- **SudokuBoard**: it models the board through a list of rows, each row being a list of SudokuCell objects. It contains methods to create a board from a file, setting a value in a cell, checking if the board is solved, etc.
- **SudokuInputReader**: it implements the reading of input commands from the user.
- **SudokuMain**: the main class of the game. Its constructor creates a random Sudoku board (from a list of predefined ones), initializes the input reader object and game state, and enables initiating the game by calling method `play()`.

Your task consists of implementing various methods:

- From class **SudokuBoard**:
 - `containsInBox(row, col, value)`. It checks if the value `value` is already present in the 3×3 box corresponding to coordinates `(row, col)`.
 - `isSolved()`. It checks if the puzzle has been solved, i.e., all cells are filled in with values between 1 and 9, with no repeated values in any row, column, or 3×3 box.
 - `toString()`. Produces a string representation of the current state of the board.
- From class **SudokuCell**:
 - `setValue(value)`. Sets the value of a (not fixed) cell.
 - `toString()`. Method is already implemented, but it needs to be reimplemented to show as an underscore when a cell is not filled.
 - `makeFixed()`. Makes a filled in cell fixed.

Your task is to implement all the above methods. You may decide to add auxiliary methods as private, or implement the required functionalities within the provided method bodies. You are not allowed to change the APIs of the classes, i.e., not change method names, number of parameters or their types. You must not modify other existing methods in the implementation. We recommend to first study the project to understand how it is organized and how the functionalities are distributed in the four different classes, before starting with the implementation.

Your solution should contain implementations of all methods. You must submit the solution through moodle, as a zip archive containing the whole project. This assignment must be solved individually. The correctness of your solution is important, as so is the quality of the code (clarity, readability, structure, etc.).